

Exemple de soumission d'activité

ÉTS - LOG430 - Architecture logicielle - Été 2026

Étudiant(e) : Chris-Emmanuel Berton

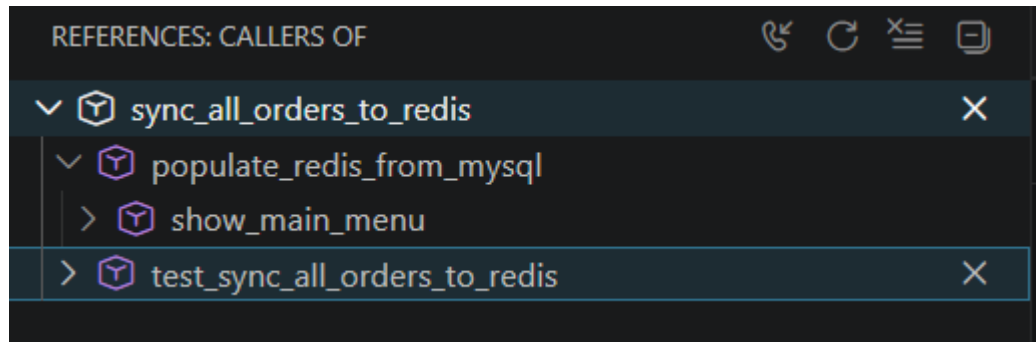
Questions

(Il est obligatoire d'ajouter du code, des captures d'écran ou des sorties de terminal pour illustrer chacune de vos réponses.)

Question 1 : Lorsque l'application démarre, la synchronisation entre Redis et MySQL est-elle initialement déclenchée par quelle méthode ? Veuillez inclure le code pour illustrer votre réponse.

Réponse Qui appelle sync La synchronisation Redis-MySQL se fait par la fonction `populate_redis_from_mysql()` puisqu'elle est la première fonction à appeler `sync_all_orders_to_redis`

```
✓ def populate_redis_from_mysql():  
    """ Populate Redis with orders from MySQL, only if Redis is empty """  
    sync_all_orders_to_redis()
```



```
REFERENCES: CALLERS OF  
✓ sync_all_orders_to_redis  
  ✓ populate_redis_from_mysql  
    > show_main_menu  
      > test_sync_all_orders_to_redis
```

Question 2 : Quelles méthodes avez-vous utilisées pour lire des données à partir de Redis ? Veuillez inclure le code pour illustrer votre réponse.

La lecture des données par le Redis se fait à partir de la fonction `get_orders_from_redis`

```
def get_orders_from_redis(limit=9999):  
    """Get last X orders"""  
    r = get_redis_conn()  
    order_ids = r.lrange("orders", 0, limit - 1)  
    print(limit)  
  
    orders = []  
    order_ids = r.zrevrange("orders", 0, limit - 1)  
  
    return [  
        r.hgetall(f"order:{order_id}")  
        for order_id in order_ids  
    ]
```

Question 3 : Quelles méthodes avez-vous utilisées pour ajouter des données dans Redis ? Veuillez inclure le code pour illustrer votre réponse.

L'ajout des données se fait par la fonction `add_order_to_redis`. Ladite fonction se fait appeler par `add_order`

```
def add_order_to_redis(order_id, user_id, total_amount, items):  
    """Insert order to Redis"""  
  
    r = get_redis_conn()  
    print(r)  
    """  
    Store order and order items in Redis  
    """  
  
    # Store main order  
    r.hset(  
        f"order:{order_id}",  
        mapping={  
            "id": order_id,  
            "user_id": user_id,  
            "total_amount": total_amount  
        }  
    )  
  
    # Store order items  
    for index, item in enumerate(items):  
  
        r.hset(  
            f"order:{order_id}:item:{index}",  
            mapping={  
                "product_id": item["product_id"],  
                "quantity": item["quantity"],  
                "unit_price": item["unit_price"]  
            }  
        )  
  
    # Keep track of newest orders  
    r.lpush("orders", order_id)
```

Question 4 : Quelles méthodes avez-vous utilisées pour supprimer des données dans Redis ? Veuillez inclure le code pour illustrer votre réponse.

La suppression des données se fait par la méthode `delete_order_from_redis`, la méthode étant appelée par `delete_order`

```
def delete_order(order_id: int):
    """Delete order in MySQL, keep Redis in sync"""
    session = get_sqlalchemy_session()
    try:
        order = session.query(Order).filter(Order.id == order_id).first()

        if order:
            session.delete(order)
            session.commit()

            # TODO: supprimer la commande à Redis
            delete_order_from_redis(order_id)
            return int(1)
        else:
            return int(0)

    except Exception as e:
        session.rollback()
        raise e
    finally:
        session.close()
```

```
def delete_order_from_redis(order_id):
    """Delete order from Redis"""
    # Delete main order hash

    r = get_redis_conn()
    print(r)

    r.delete(f"order:{order_id}")

    # Find all item keys
    item_keys = r.keys(f"order:{order_id}:item:*")

    # Delete all item hashes
    if item_keys:
        r.delete(*item_keys)

    # Remove order ID from latest orders list
    r.lrem("orders", 0, str(order_id))
```

Question 5 : Si nous souhaitions créer un rapport similaire, mais présentant les produits les plus vendus, les informations dont nous disposons actuellement dans Redis sont-elles suffisantes, ou devrions-nous chercher dans les tables sur MySQL ? Si nécessaire, quelles

informations devrions-nous ajouter à Redis ? Veuillez inclure le code pour illustrer votre réponse.

Oui : les informations sur la commande permettent de déduire autant les plus grand acheteurs en liant les id au total dépensé en commande que les articles les plus vendus en liant les id des produits à leurs quantité. Dans les deux scénarios, l'information interprétée reste le OrderItem.

```
class OrderItem(Base):
    __tablename__ = 'order_items'

    id = Column(Integer, primary_key=True, autoincrement=True)
    order_id = Column(Integer, ForeignKey('orders.id'), nullable=False)
    product_id = Column(Integer, nullable=False)
    quantity = Column(Integer, nullable=False)
    unit_price = Column(Float, nullable=False)

    # Relationship back to order
    order = relationship("Order", back_populates="order_items")
```

```
def get_highest_spending_users():
    """Get report of highest spending users"""
    r = get_redis_conn()
    keys = r.keys("order:*")
    orders = []

    for key in keys:
        if ":item:" in key:
            continue
        data = r.hgetall(key)
        if not data:
            continue
        orders.append(data)

    expenses_by_user = defaultdict(float)

    for order in orders:
        expenses_by_user[int(order['user_id'])] += float(order['total_amount'])

    highest_spending_users = sorted(
        expenses_by_user.items(),
        key=lambda item: item[1],
        reverse=True
    )
    result = []

    for user_id, total in highest_spending_users[:10]:
        result.append(
            f"<li>{user_id}: {total:.2f}$</li>"
        )

    return "".join(result)
```

```
def get_best_sellers():
    """Get report of best selling products"""

    r = get_redis_conn()

    keys = r.keys("product*:sold")

    quantities_by_product = {}

    for key in keys:
        value = r.get(key)

        if value is None:
            continue

        product_id = int(key.split(":")[1])
        quantities_by_product[product_id] = int(value)

    best_sellers = sorted(
        quantities_by_product.items(),
        key=lambda x: x[1],
        reverse=True
    )

    return "".join(
        f"<li>Produit {pid}: {qty} vendu(s)</li>"
        for pid, qty in best_sellers[:10]
    )
```

Déploiement

(Le cas échéant, décrivez votre pipeline CI/CD et ce que vous avez appris dans ce laboratoire en ce qui concerne le déploiement. Il est obligatoire d'ajouter du code, des captures d'écran ou des sorties de terminal pour illustrer votre réponse.)